

Linux实操练习手册

系统信息查看 ★★

本章收获：快速了解当前系统基本信息

实际意义：排查问题前先确认系统版本和硬件环境

练习1：查看系统内核和版本

第一步：查看完整系统信息

```
uname -a
```

预期输出：

```
Linux ubuntu 5.15.0-91-generic #101-Ubuntu SMP Tue Nov 14 13:30:08 UTC 2023
x86_64 x86_64 x86_64 GNU/Linux
```

关键信息：

- `Linux`：内核名称
- `ubuntu`：主机名
- `5.15.0-91-generic`：内核版本
- `x86_64`：系统架构（64位）

收获：`-a` 显示所有信息，`-r` 只显示内核版本

意义：安装驱动或软件时，必须确认内核版本是否兼容

练习2：查看发行版信息

第一步：查看Ubuntu版本

```
lsb_release -a
```

预期输出：

```
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.3 LTS
Release:        22.04
Codename:       jammy
```

第二步：快速查看版本文件

```
cat /etc/os-release
```

收获: `lsb_release` 最直观, `/etc/os-release` 适合脚本读取

意义: 不同Ubuntu版本软件源不同, 必须确认版本号

练习3: 查看主机名和登录信息

第一步: 查看主机名

```
hostname
```

第二步: 查看登录用户

```
who          # 谁登录了系统
uptime       # 系统运行了多久
```

预期输出:

```
14:30:00 up 3 days, 2:15, 1 user,  load average: 0.52, 0.58, 0.59
```

关键信息:

- `up 3 days`: 运行3天未重启
- `load average`: 1/5/15分钟平均负载

收获: `uptime` 看系统稳定性, 负载超过CPU核数说明繁忙

意义: 负载持续过高需扩容或优化程序

练习4: 查看命令历史记录

第一步: 查看最近使用过的命令

```
history
```

预期输出:

```
1  uname -a
2  lsb_release -a
3  hostname
...
```

收获: `history` 查看历史命令, `!n` 快速重复执行第n条

意义: 快速重复执行复杂命令, 无需重新输入

练习5: 打印文本内容

第一步: 使用echo输出简单文本

```
echo HELLO LINUX
```

常用写法：

```
echo "hello world"      # 双引号包裹字符串
echo 'hello world'      # 单引号包裹字符串
echo $HOME              # 输出变量值
echo "今天是：$(date +%Y-%m-%d)"  # 输出命令结果
```

注意事项：

```
echo hello linux        # 无引号也可以，但含空格需加引号
echo "100 + 200 = $((100+200))"  # 输出计算结果
```

收获： `echo` 输出字符串， `$(命令)` 嵌入命令结果
意义：脚本中输出提示信息、打印变量值

练习6：查看与设置系统日期

第一步：查看当前日期时间

```
date
```

预期输出：

```
Mon May 18 14:30:00 CST 2026
```

第二步：按指定格式输出日期

```
date +%Y-%m-%d          # 2026-05-18
date +%Y-%m-%d %H:%M:%S # 2026-05-18 14:30:00
```

练习7：查找文件

find：在目录树中搜索文件，最常用的文件查找命令

```
find / -name "*.zip"      # 查找根目录下所有.zip文件
find /etc -name "passwd"  # 在/etc下查找名为passwd的文件
find /home -type d        # 查找目录而非文件
find . -mtime -1          # 查找1天内修改过的文件
```

常用选项：

选项	说明	示例
<code>-name</code>	按文件名查找	<code>find . -name "*.conf"</code>
<code>-type f/d/l</code>	文件/目录/链接	<code>find / -type f -name "*.log"</code>
<code>-mtime -n</code>	n天内修改	<code>find . -mtime -7</code>

选项	说明	示例
<code>-size</code>	按大小查找	<code>find . -size +100M</code>
<code>-exec</code>	对结果执行命令	<code>find . -name "*.tmp" -exec rm {} \;</code>

常用技巧:

```
find / -name "*.txt" 2>/dev/null      # 忽略权限报错
find . -type f -empty                 # 查找空文件
```

收获: `find` 是Linux最强大的文件搜索命令，支持按名字/时间/大小等多条件
意义: 排查配置、定位日志、清理临时文件都要用到

核心命令表

命令	作用	常用选项
<code>uname -a</code>	查看系统所有信息	<code>-r</code> 内核版本 <code>-m</code> 架构
<code>lsb_release -a</code>	查看发行版信息	-
<code>cat /etc/os-release</code>	查看版本文件（脚本用）	-
<code>hostname</code>	查看主机名	-
<code>who</code>	查看登录用户	-
<code>uptime</code>	查看运行时间和负载	-
<code>history</code>	查看命令历史	<code>history n</code> 最近n条
<code>echo</code>	打印文本	<code>echo \$变量</code>
<code>date</code>	查看/设置系统日期	<code>date +%Y-%m-%d</code>

第1章 用户与组管理 ★★★

本章收获: 掌握多用户系统的权限隔离机制
实际意义: 服务器上不同人员需分配不同权限，避免误操作或数据泄露

练习1：查看当前用户

```
whoami      # 显示当前用户名
id          # 显示UID、GID和所属组
```

预期输出: `uid=1000(用户名) gid=1000(组) 组=...`

收获: 理解Linux通过UID识别用户，而非用户名
意义: 排查权限问题时，内核只认UID

练习2：创建新用户

```
sudo useradd -m -s /bin/bash testuser      # -m创建家目录
sudo passwd testuser                        # 设置密码
su - testuser                              # 切换验证
exit                                         # 返回原用户
```

收获：-m 必加，否则无家目录；-s 必加，否则环境变量错乱
意义：创建系统账号（如www-data运行网站服务）

练习3：用户组管理

```
sudo groupadd devgroup
sudo usermod -aG devgroup testuser          # -aG追加，漏-a会覆盖原有组
groups testuser                             # 验证
```

收获：用户可属于多个组，权限取并集
意义：开发组、运维组、审计组分离管理

练习4：删除用户和组

```
sudo userdel -r testuser                    # -r彻底删除
sudo groupdel devgroup
id testuser                                # 验证已删除
```

收获：-r 删家目录，否则残留垃圾文件
意义：离职人员账号及时清理，符合安全合规

第1章 核心命令表

命令	作用	必加选项
whoami	显示当前用户	-
id	显示用户ID和组	-
sudo useradd -m 用户名	创建用户	-m 创建家目录
sudo passwd 用户名	设置密码	-
sudo userdel -r 用户名	删除用户	-r 删家目录
sudo groupadd 组名	创建组	-
sudo usermod -aG 组名 用户名	加入组	-aG 追加
groups 用户名	查看所属组	-
su - 用户名	切换用户	- 带环境变量

常见问题:

- `Permission denied` → 加 `sudo`
- 忘记 `-m` → `userdel -r` 重建
- 忘记 `-a` → 用户从其他组被踢出

第2章 文件与目录管理 ★★★★★

本章收获: 理解Linux"一切皆文件"和权限模型

实际意义: 网站目录权限配置不当会导致被入侵或无法访问

练习1: 目录导航

```
pwd                # 当前位置
cd /var/log        # 绝对路径
cd ..              # 上级
cd -                # 刚才的目录
cd ~                # 主目录
```

收获: `pwd` 确认位置后再操作, 防止误删

意义: 服务器目录层级深, 导航错误可能删系统文件

练习2: 列出文件

```
ls -l              # 详细列表
ls -la             # 显示隐藏文件 (.开头)
ls -lh /var/log    # 人性化大小
ls -lt             # 按时间排序
```

关键观察: `drwxr-xr-x` → d=目录, 三组权限=所有者/组/其他人

收获: `-a` 发现隐藏配置文件 (如.ssh)

意义: 排查问题时, 病毒常伪装成隐藏文件

练习3: 创建目录和文件

```
mkdir -p project/src/main    # -p递归创建
touch project/readme.txt
ls -R project                 # 递归验证
```

收获: `-p` 一次建多级目录, 避免逐层mkdir

意义: 自动化部署时需批量创建项目结构

练习3.5：创建空文件（touch）

touch：创建空文件，或更新已有文件的访问时间

```
touch readme.txt          # 创建空文件
ls -l readme.txt          # 查看文件信息
```

常用场景：

```
touch file1.txt file2.txt file3.txt    # 同时创建多个空文件
```

收获：touch 创建空文件，mkdir 创建目录
意义：新建脚本、配置文件、日志文件时先touch占位

练习3.6：vi / nano 文本编辑器

nano（适合新手，直观简单）

```
nano readme.txt          # 用nano打开/新建文件
```

- Ctrl+O 保存 → Enter 确认 → Ctrl+X 退出
- 编辑器底部有快捷键提示，^ 代表 Ctrl

vi / vim（功能强大，服务器标配）

```
vi readme.txt            # 用vi打开/新建文件
```

三种模式：

模式	按键	说明
命令模式	默认进入	移动光标、删除、复制
插入模式	i 或 a	编辑文字
底行模式	:	保存、退出、搜索

最常用操作：

```
i          # 命令模式按 i 进入插入模式，开始编辑
ESC        # 退回命令模式
:w         # 底行模式，保存（write）
:q         # 底行模式，退出（quit）
:wq        # 保存并退出
:q!        # 强制退出不保存
```

收获：nano 适合新手快速编辑，vi 是服务器标配必须掌握
意义：所有Linux服务器都没有图形界面，改配置必须用vi/nano

练习4：复制和移动

```
cp project/readme.txt project/readme.bak
cp -r project project_backup      # -r复制目录
mv project/readme.bak project/old.txt
mv project/old.txt /tmp/         # 移动
```

收获：cp 默认不复制目录，必须 -r
意义：备份配置前先 cp 备份，改错可恢复

练习5：删除

```
rm /tmp/old.txt                  # 删文件
rm -r project_backup             # 删目录
# 或 rmdir project_backup       # 只能删空目录
```

⚠⚠⚠ 永远不要运行 ⚠⚠⚠：rm -rf / 或 rm -rf /*

收获：rm 无回收站，删除即永久
意义：生产环境建议先 mv 到/trash，定期清理

练习6：查看文件内容

```
cat /etc/passwd                 # 小文件
cat -n /etc/passwd              # 带行号
less /var/log/syslog             # 大文件分页，q退出
head -5 /etc/passwd              # 前5行
tail -5 /var/log/syslog          # 后5行（看最新日志）
```

收获：less 支持搜索（/关键词），tail 看实时日志
意义：排查故障时，用 tail 盯最新错误，用 less 搜历史

练习7：权限管理

准备：

```
touch testfile
ls -l testfile                  # 默认-rw-r--r--
```

数字法：

```
chmod 755 testfile              # rwxr-xr-x
```

数字	权限	说明
7	rwx	读+写+执行
6	rw-	读+写

数字	权限	说明
5	r-x	读+执行
4	r--	只读
0	---	无权限

符号法：

```
chmod u+x testfile      # 所有者加执行
chmod g-w testfile      # 组去掉写
chmod o=r testfile      # 其他人只读
```

改所有者：

```
sudo chown root:root testfile  # 同时改用户和组
```

收获：目录需要 `x` 才能进入，`r` 才能列出
意义：网站目录通常755，文件644，防止上传脚本被执行

练习8：创建链接

```
echo "content" > original.txt
ln original.txt hardlink.txt    # 硬链接（同inode）
ln -s original.txt softlink.txt # 软链接（快捷方式）
ls -li                         # -i看inode号
```

测试区别：

```
rm original.txt
cat hardlink.txt    # 正常（数据还在）
cat softlink.txt    # 报错（悬空链接）
```

收获：硬链接防误删，软链接方便版本切换
意义：程序升级时，改软链接指向新版本，回滚只需改链接

第2章 核心命令表

命令	作用	关键选项
<code>pwd</code>	显示当前路径	-
<code>cd</code>	切换目录	<code>..</code> 上级 <code>~</code> 家目录 <code>-</code> 刚才
<code>ls -la</code>	详细列出	<code>-l</code> 详细 <code>-a</code> 隐藏 <code>-h</code> 人性化 <code>-t</code> 时间排序
<code>mkdir -p</code>	创建目录	<code>-p</code> 递归
<code>touch</code>	创建空文件	-

命令	作用	关键选项
<code>cp -r</code>	复制目录	<code>-r</code> 递归
<code>mv</code>	移动/重命名	-
<code>rm -r</code>	删除目录	⚠️ 危险，先用 <code>ls</code> 确认
<code>cat</code>	查看小文件	<code>-n</code> 行号
<code>less</code>	分页查看	按 <code>q</code> 退出， <code>/</code> 搜索
<code>head/tail</code>	看开头/结尾	<code>-n</code> 行数， <code>tail -f</code> 实时
<code>chmod 755</code>	数字改权限	7=rwx,6=rw-,5=r-x,4=r--
<code>chmod u+x</code>	符号改权限	u用户g组o其他
<code>chown 用户:组</code>	改所有者	-
<code>ln</code>	硬链接	同inode，不能跨分区
<code>ln -s</code>	软链接	常用，类似快捷方式

第2章 综合练习

```
cd ~
mkdir -p mysite/{html,css,js,logs}
touch mysite/html/index.html
echo "console.log('hello')" > mysite/js/app.js
chmod 755 mysite/html
chmod 644 mysite/html/index.html
chmod 700 mysite/logs
ln -s mysite/logs ~/loglink
ls -la mysite/
```

收获：权限规划原则：目录755，文件644，敏感目录700
意义：Web服务器标准权限配置，防止目录遍历和文件泄露

第3章 磁盘存储管理 ★

本章收获：理解Linux磁盘挂载机制
实际意义：服务器扩容、数据迁移、备份存储都需要挂载操作

练习1：查看磁盘信息

```
lsblk          # 磁盘分区结构
lsblk -f       # 显示文件系统类型
df -h          # 磁盘使用空间
```

识别: `sda` =磁盘, `sda1/2/3` =分区, `/` =挂载点, `[SWAP]` =交换分区

收获: `/` 根分区满了会导致系统崩溃

意义: 监控 `/var/log` 防止日志占满磁盘

练习2: 查看目录大小

```
du -sh ~ # 家目录总大小
du -h --max-depth=1 /etc 2>/dev/null | sort -hr | head -5
```

收获: `du` 慢但准, `df` 快但看整体

意义: 定位大文件占用, 清理 `/var/log` 或 `/tmp`

练习3: 挂载与卸载

创建虚拟磁盘:

```
dd if=/dev/zero of=~/.fake.disk bs=1M count=100 # 创建100M空文件
sudo mkfs.ext4 ~/.fake.disk # 格式化
```

挂载使用:

```
sudo mkdir -p /mnt/fake
sudo mount -o loop ~/.fake.disk /mnt/fake # -o loop关键
df -h | grep fake # 验证
sudo touch /mnt/fake/test.txt # 使用
```

卸载:

```
sudo umount /mnt/fake # 必须卸载后再删文件
rm ~/.fake.disk
sudo rmdir /mnt/fake
```

收获: `umount` 前确保无程序占用该目录

意义: U盘/移动硬盘必须卸载再拔, 否则数据损坏

练习4: /etc/fstab配置

```
cat /etc/fstab # 开机自动挂载配置
sudo blkid # 查看UUID
```

6列含义: 设备|挂载点|文件系统类型|选项|dump标记|fsck顺序

收获: 用UUID比设备名 (如 `/dev/sda1`) 更可靠, 磁盘顺序变也不影响

意义: 配置错误会导致系统无法启动, 修改前必须备份

第3章 核心命令表

命令	作用	关键选项
<code>lsblk</code>	查看磁盘分区	<code>-f</code> 显示文件系统
<code>df -h</code>	查看磁盘空间	<code>-h</code> 人性化 <code>-T</code> 显示类型
<code>du -sh</code>	查看目录大小	<code>-s</code> 汇总 <code>-h</code> 人性化
<code>sudo mount</code>	挂载	<code>-o loop</code> 挂载文件
<code>sudo umount</code>	卸载	-
<code>sudo blkid</code>	查看UUID	-

第4章 软件包管理 ★★★★★

本章收获：掌握APT包管理系统
实际意义：服务器软件安装、更新、漏洞修复的标准流程

练习1：更换软件源

方式一：tee 覆盖写入（适合脚本自动化）

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak      # 备份
sudo tee /etc/apt/sources.list << 'EOF'
deb http://mirrors.aliyun.com/ubuntu/ jammy main restricted universe multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-updates main restricted universe
multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-security main restricted universe
multiverse
EOF
sudo apt update                                              # 生效
```

方式二：vi 编辑（服务器标配）

```
sudo vi /etc/apt/sources.list
```

进入后按 `i` 进入插入模式，删除原内容，粘贴上方阿里云源内容

保存退出：`ESC` → `:wq`

方式三：nano 编辑（适合新手）

```
sudo nano /etc/apt/sources.list
```

删除原内容，粘贴阿里云源内容，`Ctrl+O` 保存 → `Enter` 确认 → `Ctrl+X` 退出

收获：国内镜像加速下载，教育网可用清华/中科大源
意义：国外源慢且可能断，生产环境必须换国内源

练习2：安装软件

```
apt search nginx          # 搜索
apt show nginx            # 详情
sudo apt install nginx -y # 安装（-y自动确认）
which nginx              # 验证安装路径
dpkg -l | grep nginx     # 查看已安装包
sudo service nginx start  # 启动（传统service命令）
sudo service nginx status # 查看状态
# 浏览器访问 http://localhost 验证
```

收获：apt 自动解决依赖，dpkg -l 查看已安装软件
意义：批量部署服务器时，用脚本执行 apt install

练习3：卸载和清理

```
sudo apt remove nginx -y # 卸载（保留配置）
sudo apt purge nginx -y  # 彻底卸载（推荐）
sudo apt autoremove -y   # 清理残留依赖
sudo apt clean           # 清理下载缓存
```

收获：purge 比 remove 干净，autoremove 防依赖堆积
意义：长期运行的服务器，定期 autoremove 释放空间

练习4：升级系统

```
sudo apt update          # 更新列表
apt list --upgradeable   # 查看可升级
sudo apt upgrade -y      # 安全升级
```

收获：update 只更新列表，upgrade 才升级软件
意义：安全漏洞需及时升级，但生产环境建议先测试

第4章 核心命令表

命令	作用	关键选项
sudo apt update	更新软件列表	-
apt search	搜索软件	-
apt show	查看详情	-
sudo apt install	安装	-y 自动确认

命令	作用	关键选项
<code>sudo apt purge</code>	彻底卸载	<code>-y</code> 自动确认
<code>sudo apt autoremove</code>	清理依赖	<code>-y</code> 自动确认
<code>sudo apt clean</code>	清理缓存	-
<code>sudo apt upgrade</code>	升级软件	<code>-y</code> 自动确认
<code>apt list --upgradeable</code>	查看可升级	-
<code>dpkg -l</code>	查看已安装	<code> grep</code> 过滤
<code>sudo service 服务名 start/stop/status</code>	管理服务	-

第5章 进程与系统管理 ★★

本章收获：掌握进程监控和任务自动化
实际意义：服务器性能瓶颈排查、定时备份、自动运维

练习1：查看和结束进程

```
ps aux                # 所有进程
ps aux | grep bash    # 过滤查找
```

结束进程：

```
kill PID              # 正常终止（PID从ps aux获取）
kill -9 PID           # 强制终止
killall 进程名        # 按名称结束
```

收获：`kill -9`是最后手段，正常程序应先`kill`
意义：进程卡死时，先`kill`，无效再`kill -9`

练习2：实时监控

```
top                    # 实时进程监控
```

操作：

- 按 `M`：内存排序
- 按 `P`：CPU排序（默认）
- 按 `q`：退出

```
free -h               # 查看内存
df -h                 # 查看磁盘
```

收获: top 看实时, free/df 看整体
意义: 内存不足时, top 找占用大的进程杀掉或重启

练习3：定时任务

```
crontab -e                                # 编辑（首次选nano=1）
```

添加：

```
* * * * * date >> ~/crontest.log        # 每分钟执行
```

```
crontab -l                                # 查看任务
cat ~/crontest.log                        # 等待2分钟验证
crontab -e                                # 删除测试行
rm ~/crontest.log
```

时间格式：

符号	含义
<code>*/5</code>	每5分钟
<code>0 *</code>	每小时
<code>0 2</code>	每天2点
<code>0 0 * * 0</code>	每周日

收获: crontab 最小单位分钟，系统级任务在 /etc/cron.d/
意义: 定时备份、日志清理、健康检查自动化

第5章 核心命令表

命令	作用	关键选项
<code>ps aux</code>	查看所有进程	<code> grep</code> 过滤
<code>kill PID</code>	结束进程	<code>-9</code> 强制
<code>killall</code>	按名称结束	-
<code>top</code>	实时监控	<code>M</code> 内存 <code>P</code> CPU <code>q</code> 退出
<code>free -h</code>	查看内存	<code>-h</code> 人性化
<code>df -h</code>	查看磁盘	<code>-h</code> 人性化
<code>crontab -e</code>	编辑定时任务	-
<code>crontab -l</code>	查看任务	-

第5章 综合练习

```
# 创建每分钟记录的任务
crontab -e
# 添加: * * * * * echo "$(date) system check" >> ~/check.log

# 验证
sleep 120                # 等待2分钟
cat ~/check.log

# 清理
crontab -e                # 删除该行
rm ~/check.log
```

收获: `crontab` 是运维自动化的基础

意义: 数据库定时备份、日志切割、监报告警都依赖它

第6章 systemctl 服务管理 ★★

本章收获: 掌握 Linux 服务管理机制, 理解开机自启动原理

实际意义: 服务器运行状态监控、故障自愈配置、运维自动化基础

练习1: 查看系统所有服务状态

【生产环境使用场景】

服务器重启后或日常巡检时, 运维人员需要快速了解"当前有哪些服务在运行、哪些服务处于异常状态", 以判断系统是否正常。这是所有运维排查的第一步。

第一步: 查看所有服务列表

```
systemctl list-units --type=service --all
```

含义: `list-units` 列出所有单元, `--type=service` 过滤服务类型, `--all` 包含未启动的

第二步: 只查看正在运行的服务

```
systemctl list-units --type=service --state=running
```

第三步: 查看服务详细信息 (含启动时间、内存占用)

```
systemctl status nginx
```

✅ 验证成功的表现:

- 能看到 `Active: active (running)` 或 `inactive (dead)` 等状态
- 能看到 Main PID、Memory、Tasks 等运行时信息

【收获】 `systemctl` 是 Ubuntu/Debian 系 Linux 服务管理的标准工具

【意义】 上线新服务前, 必须确认端口是否被占用、服务是否成功拉起

练习2：更换阿里云软件源（后续练习前提）

【生产环境使用场景】

在国内服务器、教学机房或内网环境中，默认官方源下载速度慢甚至无法访问，通常需要替换为国内镜像源（如阿里云、清华源）以提高软件安装效率。这是所有服务器初始化后第一步操作。

第一步：备份原有软件源

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.bak
cat /etc/apt/sources.list    # 查看当前软件源内容
```

第二步：替换为阿里云镜像源（Ubuntu 22.04 / jammy）

有两种方式，任选其一练习：

方式一：tee 一步覆盖写入（推荐生产环境）

```
sudo tee /etc/apt/sources.list << 'EOF'
deb http://mirrors.aliyun.com/ubuntu/ jammy main restricted universe multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-updates main restricted universe
multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-security main restricted universe
multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-backports main restricted universe
multiverse
EOF
```

方式二：vi / nano 手动编辑（练习文本编辑）

```
# 用 vi 编辑
sudo vi /etc/apt/sources.list

# 或用 nano 编辑（更直观，推荐新手）
sudo nano /etc/apt/sources.list
```

用 vi/nano 打开文件后，删除原有所有行，粘贴以下内容（vi 中按 **i** 进入插入模式，nano 中 Ctrl+O 可直接写入）：

```
deb http://mirrors.aliyun.com/ubuntu/ jammy main restricted universe multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-updates main restricted universe
multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-security main restricted universe
multiverse
deb http://mirrors.aliyun.com/ubuntu/ jammy-backports main restricted universe
multiverse
```

vi 保存退出：ESC → :wq

nano 保存退出：Ctrl+O 回车 → Ctrl+X

第三步：更新软件包索引

```
sudo apt update
```

✅ 验证成功的表现：

- `apt update` 无报错，结束时显示 "Reading package lists... Done"
- `apt update` 输出中能看到 `mirrors.aliyun.com` 相关地址

【收获】换源后必须 `apt update` 才能使新源生效；`tee` 适合脚本自动化，`vi/nano` 适合手动细粒度编辑

【意义】国内服务器必做操作，否则 `apt install` 经常失败或极慢

练习3：管理单个服务（nginx）

【生产环境使用场景】

部署 Web 站点、启动 API 服务、关闭临时测试进程时，都需要启动/停止/重启服务。`systemctl` 是这些操作的标准入口。

第一步：安装 nginx（如未安装，确保软件源已更换）

```
sudo apt update                # 更新软件包索引（使新换的阿里云源生效）
sudo apt install nginx -y      # 安装 nginx web 服务器
```

✅ `which nginx` 确认可执行文件存在

第二步：启动 nginx 服务

```
sudo systemctl start nginx      # 启动 nginx 服务
sudo systemctl status nginx     # 查看 nginx 服务状态，确认是否 running
```

✅ 显示 `Active: active (running)` 表示启动成功

第三步：停止 nginx 服务

```
sudo systemctl stop nginx       # 停止 nginx 服务
sudo systemctl status nginx     # 查看 nginx 服务状态，确认是否已停止
```

✅ 显示 `Active: inactive (dead)` 表示已停止

第四步：重启 vs reload 对比

```
sudo systemctl restart nginx    # 完全停止再启动，PID 会变化
sudo systemctl reload nginx     # 重新加载配置文件，PID 不变（nginx 专有）
sudo systemctl status nginx     # 查看状态，确认 restart 后 PID 是否变化
```

✅ `restart` 后 Main PID 数字会变化

【收获】`start/stop/restart/reload` 适用于不同场景，`reload` 更平滑

【意义】生产环境更新配置后用 `reload` 无损加载，改动大则 `restart`

练习4：设置开机自启动

【生产环境使用场景】

服务器断电重启后，业务进程必须能自动拉起，否则需要人工登录操作。配置开机自启动是保障业务连续性的基础。

第一步：查看服务当前是否设为开机自启动

```
systemctl is-enabled nginx
```

输出 `enabled` 表示开机自启动，`disabled` 表示不会自启动

第二步：禁用开机自启动

```
sudo systemctl disable nginx      # 取消 nginx 开机自启动
systemctl is-enabled nginx        # 验证是否已取消，输出应为 disabled
```

✓ 输出变为 `disabled`

第三步：重新启用开机自启动

```
sudo systemctl enable nginx      # 设置 nginx 开机自启动
systemctl is-enabled nginx       # 验证是否已开启，输出应为 enabled
```

✓ 输出变为 `enabled`

第四步：重启验证（选做）

```
sudo reboot    # 重启虚拟机
```

✓ `is-active` 输出 `active` 表示重启后自动运行

【收获】 `enable/disable` 控制开机自启动，`is-active` 只看当前是否运行

【意义】 核心业务（`nginx/mysql/docker`）必须 `enable`，临时工具则不需要

练习5：查看服务日志

【生产环境使用场景】

服务异常退出、启动失败、访问报错时，首先要看日志。`journalctl` 是 `systemctl` 配套的日志查看工具，能看到服务每次启动的完整输出。

第一步：查看 nginx 最近 20 条日志

```
sudo journalctl -u nginx --no-pager -n 20
```

第二步：实时追踪日志（模拟访问触发日志）

```
# 终端1：实时监控 nginx 日志
sudo journalctl -u nginx -f

# 终端2：访问本机 nginx（触发日志）
curl http://localhost
```

退出实时监控：Ctrl + C

第三步：查看指定时间段的日志

```
sudo journalctl -u nginx --since "10 minutes ago" # 查看最近 10 分钟的 nginx 日志
sudo journalctl -u nginx --since today             # 查看今天所有 nginx 日志
```

✅ 能看到带时间戳的每条日志记录

【收获】 `-f` 实时追踪, `-n` 指定行数, `--since` 指定时间范围

【意义】 排查故障时 `journalctl` 是第一手材料, 比 `/var/log` 更实时

练习6: 创建一个自定义 systemd 服务

【生产环境使用场景】

将自己写的脚本（如数据备份、定时采集）注册为系统服务，可以通过 `systemctl` 统一管理启停和日志，这是 Linux 运维进阶的必备技能。

第一步：创建一个测试用 Shell 脚本

```
mkdir -p ~/bin
cat > ~/bin/hello.sh << EOF
#!/bin/bash
while true; do
    echo "$(date): hello from systemd service" >> /tmp/hello.log
    sleep 5
done
EOF
chmod +x ~/bin/hello.sh
```

第二步：编写 systemd 服务单元文件

```
sudo tee /etc/systemd/system/hello.service << EOF
[Unit]
Description=Hello Custom Service
After=network.target

[Service]
ExecStart=/root/bin/hello.sh
Restart=always
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
```

第三步：重载 systemd 配置并启动服务

```
sudo systemctl daemon-reload # 重载 systemd 配置（修改 .service 文件后必须执行）
sudo systemctl start hello    # 启动 hello 服务
sudo systemctl status hello    # 查看 hello 服务运行状态
```

✅ 显示 `Active: active (running)`

✅ `cat /tmp/hello.log` 有内容追加

第四步：设置开机自启动并验证

```
sudo systemctl enable hello      # 设置 hello 服务开机自启动
sudo systemctl stop hello        # 停止 hello 服务（测试用）
sudo systemctl start hello       # 重新启动 hello 服务
```

✔ `systemctl is-enabled hello` 输出 `enabled`

第五步：清理（删除测试服务）

```
sudo systemctl stop hello        # 停止 hello 服务
sudo systemctl disable hello     # 取消 hello 开机自启动
sudo rm /etc/systemd/system/hello.service  # 删除 hello 服务单元文件
sudo systemctl daemon-reload    # 重载 systemd 配置，使删除生效
rm ~/bin/hello.sh               # 删除测试脚本
rm /tmp/hello.log               # 删除测试日志
```

- 【收获】 `daemon-reload` 每次改 `.service` 文件后都要执行
- 【意义】 自己写的脚本注册为服务后，就能用 `systemctl` 统一管理生命周期

第4章 核心命令表

命令	作用	关键选项
<code>systemctl list-units --type=service</code>	列出所有服务	<code>--state=running</code> 只看运行中
<code>systemctl status 服务名</code>	查看服务详细状态	含 PID/内存/启动时间
<code>systemctl start 服务名</code>	启动服务	-
<code>systemctl stop 服务名</code>	停止服务	-
<code>systemctl restart 服务名</code>	重启服务（完全重载）	-
<code>systemctl reload 服务名</code>	重新加载配置（平滑）	PID 不变
<code>systemctl is-active 服务名</code>	查看当前是否运行	返回 active/inactive
<code>systemctl is-enabled 服务名</code>	查看是否开机自启	返回 enabled/disabled
<code>systemctl enable 服务名</code>	设为开机自启动	-
<code>systemctl disable 服务名</code>	取消开机自启动	-
<code>systemctl daemon-reload</code>	重载服务配置	改 <code>.service</code> 文件后必执行
<code>journalctl -u 服务名</code>	查看服务日志	<code>-u</code> 指定服务 <code>-f</code> 实时

第4章 常见问题

- **Q: systemctl start 失败，提示 "Unit not found"** → 检查服务名是否正确，nginx 服务名是 `nginx` 而不是 `nginx.service`
- **Q: 服务一直 restart** → 检查脚本是否有语法错误，手动运行 ExecStart 确认
- **Q: 日志看不到内容** → journalctl 默认从本次 systemd 启动后开始记，多次启动前的日志用 `journalctl --since "1 hour ago"`
- **Q: 改了 .service 文件不生效** → 忘记 `daemon-reload`，每次修改后必须执行

第7章 Shell 编程基础 ★★

本章收获：掌握 Shell 变量、条件判断、循环结构，写出可执行的自动化脚本

实际意义：批量处理文件、定时自动巡检、数据清洗、运维自动化

练习1：第一个 Shell 脚本（Hello World）

【生产环境使用场景】

学习任何编程语言的第一步都是输出 Hello World。Shell 脚本的编写流程是：写脚本 → 加执行权限 → 运行。这是所有自动化运维的基础。

第一步：创建脚本文件

```
mkdir -p ~/shell_practice      # 创建练习目录
vi ~/shell_practice/hello.sh   # 用 vi 编辑器打开脚本文件
```

按 `i` 进入插入模式，输入以下内容：

```
#!/bin/bash
# 我的第一个 shell 脚本
echo "Hello world!"
echo "当前用户是：$(whoami)"
echo "今天日期是：$(date +%Y-%m-%d)"
```

保存退出：`ESC` → `:wq`

第二步：添加执行权限并运行

```
chmod +x ~/shell_practice/hello.sh  # 给脚本添加执行权限
bash ~/shell_practice/hello.sh      # 用 bash 运行脚本
```

✅ 终端输出 Hello World! 和当前用户名、日期

【收获】 `#!/bin/bash` 叫 shebang，指定脚本由哪个解释器执行，不可省略

【意义】 `$(命令)` 是命令替换，将命令输出作为值使用，是 Shell 最常用的语法之一

练习2：变量与用户输入

【生产环境使用场景】

编写备份脚本时，需要指定源目录和目标目录；写日志清理脚本时，需要传入保留天数。变量和 read 是 Shell 脚本参数化的基础。

第一步：变量赋值与输出

```
NAME="world"           # 等号两侧不能有空格
AGE=25
echo "Hello $NAME"     # $变量名 取值
echo "明年 age+1 = $((AGE + 1))" # 算术运算要用 $(( ))
```

第二步：接收用户输入

```
#!/bin/bash
echo "请输入你的名字： "
read USERNAME          # 读取用户输入并存入 USERNAME
echo "你好 $USERNAME，欢迎！ "

echo "请输入一个数字： "
read NUM
echo "你输入的数字是 $NUM，乘以 2 = $((NUM * 2))"
```

运行测试：

```
bash ~/shell_practice/variable.sh
```

✅ 能正常显示变量值，能接收键盘输入并处理

【收获】 变量名=值（无空格），\$变量名 取值，\$((expr)) 做算术，read 读输入

【意义】 变量让脚本从"写死"变成"通用"，read 让脚本能交互

练习3：expr 整数计算

【生产环境使用场景】

计算磁盘使用率、统计文件数量、计算百分比时，整数运算是基础。expr 是 Shell 原生的整数计算工具，兼容性好。

基本运算：

```
expr 10 + 5           # 加法，输出 15（注意：+ 两边必须有空格）
expr 10 - 5           # 减法，输出 5
expr 10 \* 5           # 乘法，输出 50（* 需要转义）
expr 10 / 5           # 除法，输出 2
expr 10 % 3           # 取余，输出 1
```

综合练习 — 计算当前目录文件数量：

```
#!/bin/bash
COUNT=$(ls -l | wc -l)    # 统计当前目录文件数量
echo "当前目录共有 $COUNT 个文件"

# 计算 1GB = 多少 KB
KB=$(expr 1024 \* 1024)
echo "1GB = $KB KB"
```

【收获】expr 运算时操作数之间必须有空格，* 需要加 \ 转义

【意义】加减乘除取模是所有监控脚本和统计脚本的基础运算

练习4：条件判断 if ★★

【生产环境使用场景】

服务器磁盘空间低于阈值时告警、服务进程不存在时自动重启、备份文件不存在时报错——这些"如果...则..."的逻辑都用 if 判断实现。

基本 if 语法：

```
#!/bin/bash
NUM=10
if [ $NUM -gt 5 ]; then      # -gt: greater than
    echo "$NUM > 5, 条件成立"
fi
```

if-else 语法：

```
#!/bin/bash
AGE=20
if [ $AGE -ge 18 ]; then    # -ge: greater than or equal
    echo "已成年"
else
    echo "未成年"
fi
```

if-elif-else (判断成绩等级)：

```
#!/bin/bash
SCORE=85
if [ $SCORE -ge 90 ]; then
    echo "优秀"
elif [ $SCORE -ge 80 ]; then
    echo "良好"
elif [ $SCORE -ge 60 ]; then
    echo "及格"
else
    echo "不及格"
fi
```

文件类型判断 (实战最常用)：


```
#!/bin/bash
FILE="/etc/passwd"
if [ -f "$FILE" ]; then          # -f: 普通文件存在
    echo "$FILE 存在，是普通文件"
elif [ -d "$FILE" ]; then       # -d: 目录存在
    echo "$FILE 存在，是目录"
else
    echo "$FILE 不存在"
fi
```

常用判断选项汇总：

选项	含义
<code>-f 文件</code>	普通文件存在
<code>-d 目录</code>	目录存在
<code>-e 文件/目录</code>	文件或目录存在
<code>-r 文件</code>	文件可读
<code>-w 文件</code>	文件可写
<code>-x 文件</code>	文件可执行
<code>-z 字符串</code>	字符串为空
<code>-n 字符串</code>	字符串非空
<code>A = B / A != B</code>	字符串相等/不等
<code>-lt / -le / -gt / -ge / -eq / -ne</code>	整数比较

【收获】 `[ ]` 括号内，两侧必须留空格；条件之间用 `&&` 或 `||` 连接

【意义】 `if` 判断是所有"异常检测 + 自动修复"脚本的核心逻辑

练习5：for 循环 ★★

【生产环境使用场景】

批量重命名 100 个文件、批量创建 10 个用户、遍历日志目录逐个压缩——这些"对多个对象执行相同操作"的场景，就是 `for` 循环的典型应用。

语法一：遍历数字序列

```
#!/bin/bash
# 打印 1 到 5
for i in 1 2 3 4 5; do
    echo "第 $i 次循环"
done
```

语法二：用 `seq` 生成序列（最常用）

```
#!/bin/bash
SUM=0
for i in $(seq 1 100); do    # 1 到 100
    SUM=$((SUM + i))
done
echo "1+2+...+100 = $SUM"
```

✅ 输出 5050

语法三：遍历文件列表

```
#!/bin/bash
for file in /etc/*.conf; do    # 遍历 /etc 下所有 .conf 文件
    echo "找到配置文件: $file"
done
```

语法四：批量操作示例（批量创建测试用户）

```
#!/bin/bash
for user in user01 user02 user03; do
    sudo useradd -m "$user" 2>/dev/null
    echo "已创建用户: $user"
done
id user01    # 验证
```

清理：

```
for user in user01 user02 user03; do
    sudo userdel -r "$user" 2>/dev/null    # 删除用户及其家目录
done
```

【收获】 `for 变量 in 列表; do ... done`，`seq` 是生成整数序列的利器

【意义】 批量自动化是 Shell 脚本在生产环境中最核心的价值

练习6：while 循环

【生产环境使用场景】

等待某个服务启动完成（轮询检查）、从文件逐行读取数据处理、无限循环守护进程——这些场景用 while 循环比 for 更合适。

语法：条件为真时重复执行

```
#!/bin/bash
# 猜数字游戏
SECRET=7
while true; do          # 无限循环
    echo "请输入一个数字（0-9）："
    read guess
    if [ "$guess" -eq "$SECRET" ]; then
        echo "恭喜猜对了！"
        break          # 猜对后退出循环
    else
        echo "猜错了，再试一次"
    fi
done
```

逐行读取文件（统计 /etc/passwd 行数）：

```
#!/bin/bash
COUNT=0
while read line; do
    COUNT=$((COUNT + 1))
done < /etc/passwd
echo "/etc/passwd 共有 $COUNT 行"
```

✅ 输出行数等于 `wc -l /etc/passwd` 的结果

【收获】 while 适合"不知道要循环多少次"的场景，for 适合"已知数量"的场景

【意义】 while 循环常用于"等待某个条件满足"的监控脚本

练习7：Shell 综合练习 — 自动巡检脚本

【生产环境使用场景】

这是真实的运维自动化场景：写一个脚本，自动检查 CPU 使用率、内存占用、磁盘空间、nginx 进程是否存活，并将结果写入日志文件。

编写脚本：

```
#!/bin/bash
# auto_check.sh - 服务器自动巡检脚本
# 用法：bash auto_check.sh

LOG_FILE="/tmp/check_$(date +%Y%m%d_%H%M%S).log"

echo "===== 服务器巡检报告 =====" >> $LOG_FILE
echo "巡检时间: $(date)" >> $LOG_FILE
echo "" >> $LOG_FILE

# 1. 检查 nginx 进程是否存在
if ps aux | grep -v grep | grep nginx > /dev/null; then
    echo "[OK] nginx 进程运行中" >> $LOG_FILE
else
    echo "[WARN] nginx 进程不存在" >> $LOG_FILE
fi
```

```
# 2. 检查磁盘空间（根分区使用率）
DISK_USAGE=$(df -h / | tail -1 | awk '{print $5}' | tr -d '%')
echo "根分区使用率: ${DISK_USAGE}%" >> $LOG_FILE
if [ "$DISK_USAGE" -gt 80 ]; then
    echo "[WARN] 磁盘使用率超过 80%!" >> $LOG_FILE
else
    echo "[OK] 磁盘空间正常" >> $LOG_FILE
fi

# 3. 检查内存使用率
MEM_USAGE=$(free | grep Mem | awk '{print int($3/$2 * 100)}')
echo "内存使用率: ${MEM_USAGE}%" >> $LOG_FILE

echo "===== 巡检结束 =====" >> $LOG_FILE
cat $LOG_FILE
```

运行并验证：

```
chmod +x ~/shell_practice/auto_check.sh    # 给巡检脚本添加执行权限
bash ~/shell_practice/auto_check.sh        # 运行自动巡检脚本
```

- ✓ 日志包含 nginx 状态、磁盘使用率、内存使用率
- ✓ `cat /tmp/check_*.log` 能找到生成的巡检报告

【收获】 综合脚本 = 变量 + 条件判断 + 循环 + 命令替换 + 重定向
 【意义】 这是运维自动化最基础的形态，复杂场景再加入 crontab 定时执行

第5章 核心命令表

语法/命令	作用	示例
<code>#!/bin/bash</code>	Shell 脚本声明行 (shebang)	<code>#!/bin/bash</code>
<code>变量名=值</code>	定义变量（等号两边无空格）	<code>NAME="linux"</code>
<code>\$变量名</code>	取值	<code>echo \$NAME</code>
<code>\$((expr))</code>	整数算术运算	<code>\$((10 + 5))</code> → 15
<code>expr A * B</code>	整数乘法（* 需转义）	<code>expr 3 * 4</code> → 12
<code>read 变量</code>	读取用户输入	<code>read NAME</code>
<code>\$(命令)</code>	命令替换	<code>\$(date +%Y)</code>
<code>if [条件]; then ... fi</code>	条件判断	<code>if [\$a -gt 5]; then</code>
<code>for i in 列表; do ... done</code>	for 循环	<code>for i in \$(seq 1 10)</code>
<code>while [条件]; do ... done</code>	while 循环	<code>while true</code>

语法/命令	作用	示例
<code>break</code>	跳出循环	-
<code>echo "text" > 文件</code>	覆盖写入	<code>echo "ok" > result.txt</code>
<code>echo "text" >> 文件</code>	追加写入	<code>echo "ok" >> result.txt</code>

第7章 常见问题

- **Q: 脚本报错 "Syntax error"** → 检查 `[ ]` 括号内是否有空格, `if/then/fi` 是否配对
- **Q: 变量值为空** → 变量赋值时等号两边不能有空格, 应该 `NAME=value` 而不是 `NAME = value`
- **Q: expr 乘法报错** → `*` 需要转义为 `\*`, 或者直接用 `$(a * b)` 更方便
- **Q: for 循环中变量值不变** → `for i in $(seq 1 10)` 如果 `seq` 结果为空, 检查命令是否正确
- **Q: 脚本不执行直接退出** → 先 `chmod +x` 加执行权限, 或用 `bash 脚本名` 运行

综合练习：systemctl + Shell 联用

【场景描述】

编写一个 Shell 脚本, 注册为 systemd 服务, 实现以下功能: 每分钟检查一次 nginx 是否在运行, 如果 nginx 挂了, 自动重启并记录日志。

第一步：编写监控脚本

```
#!/bin/bash
# monitor_nginx.sh - nginx 保活监控脚本
LOG="/var/log/nginx_monitor.log"

if ps aux | grep -v grep | grep nginx > /dev/null; then
    echo "$(date): nginx 正常运行" >> $LOG
else
    echo "$(date): nginx 已停止, 尝试重启..." >> $LOG
    systemctl start nginx >> $LOG 2>&1
    sleep 2
    if ps aux | grep -v grep | grep nginx > /dev/null; then
        echo "$(date): nginx 重启成功" >> $LOG
    else
        echo "$(date): nginx 重启失败!" >> $LOG
    fi
fi
```

第二步：注册为 systemd 服务（使用 timer 触发）

注: timer 是 systemd 的定时器, 比 crontab 更可靠

```
sudo tee /etc/systemd/system/nginx-monitor.service << EOF
[Unit]
Description=Nginx Auto Monitor Service

[Service]
ExecStart=/bin/bash /root/shell_practice/monitor_nginx.sh
Type=oneshot
EOF
```

```
sudo tee /etc/systemd/system/nginx-monitor.timer << EOF
[Unit]
Description=Run nginx-monitor every minute

[Timer]
OnBootSec=1min
OnUnitActiveSec=1min
Unit=nginx-monitor.service

[Install]
WantedBy=timers.target
EOF
```

第三步：启动定时器并验证

```
chmod +x ~/shell_practice/monitor_nginx.sh      # 给监控脚本添加执行权限
sudo systemctl daemon-reload                    # 重载 systemd 配置
sudo systemctl enable --now nginx-monitor.timer  # 启用定时器并立刻启动
sudo systemctl status nginx-monitor.timer        # 查看定时器状态
```

✅ Active: active (running)

```
# 等待约 1 分钟后查看日志
cat /var/log/nginx_monitor.log
```

第四步：清理

```
sudo systemctl stop nginx-monitor.timer      # 停止定时器服务
sudo systemctl stop nginx-monitor.service    # 停止监控服务
sudo systemctl disable nginx-monitor.timer   # 取消定时器开机自启动
sudo rm /etc/systemd/system/nginx-monitor.service # 删除 service 文件
sudo rm /etc/systemd/system/nginx-monitor.timer  # 删除 timer 文件
sudo systemctl daemon-reload                 # 重载 systemd 配置
```

【收获】systemd timer 比 crontab 更可靠，服务挂了会自动重启，日志永久保存

【意义】这是"监控 + 自动恢复"的最简生产模型，理解后可扩展为任意服务保活

附录：核心命令速查

systemctl 核心命令 ★★

命令	作用
<code>systemctl status 服务名</code>	查看服务运行状态
<code>systemctl start / stop 服务名</code>	启动 / 停止服务
<code>systemctl restart 服务名</code>	重启服务
<code>systemctl enable / disable 服务名</code>	开机自启 / 取消
<code>systemctl is-active / is-enabled 服务名</code>	检查运行/自启状态
<code>systemctl daemon-reload</code>	重载服务配置
<code>journalctl -u 服务名 -f</code>	实时查看服务日志
<code>systemctl list-units --type=service</code>	列出所有服务

Shell 编程核心语法

语法	说明
<code>#!/bin/bash</code>	脚本声明，必须是第一行
<code>NAME="value"</code>	定义变量
<code>\$NAME</code>	引用变量
<code>\$((expr))</code>	整数运算
<code>expr a * b</code>	乘法（需转义）
<code>read NAME</code>	读取输入
<code>[\$a -gt \$b]</code>	整数比较（-gt/-lt/-ge/-le/-eq/-ne）
<code>[-f file]</code>	文件存在判断（-f/-d/-e/-r/-w/-x）
<code>if [条件]; then ... fi</code>	if 条件语句
<code>for i in 列表; do ... done</code>	for 循环
<code>while [条件]; do ... done</code>	while 循环
<code>break</code>	跳出循环
<code>echo "text" > file</code>	覆盖写入文件
<code>echo "text" >> file</code>	追加写入文件
<code>\$(command)</code>	命令替换

第8章 Shell 正则表达式与三目运算符

本章收获：掌握 Shell 中正则表达式的用法，理解三目运算符的简写逻辑

实际意义：日志分析、配置提取、数据清洗、脚本中简化 if 判断

练习1：认识正则表达式与 grep

【生产环境使用场景】

排查 nginx 日志时，想找出所有包含特定 IP、特定时间或错误关键词的行；分析 /etc/passwd 时，想匹配符合某种模式的账号。正则表达式就是“带通配符的精确搜索”，比普通字符串过滤强大得多。

第一步：准备测试文件

```
mkdir -p ~/shell_practice
cat > ~/shell_practice/access.log << 'EOF'
192.168.1.10 - - [10/May/2026:10:12:34 +0800] "GET /index.html HTTP/1.1" 200 2326
192.168.1.15 - - [10/May/2026:10:12:35 +0800] "GET /login HTTP/1.1" 200 1024
10.0.0.88 - - [10/May/2026:10:12:36 +0800] "POST /api/login HTTP/1.1" 401 512
192.168.1.10 - - [10/May/2026:10:12:40 +0800] "GET /admin HTTP/1.1" 403 128
192.168.1.20 - - [10/May/2026:10:13:01 +0800] "GET /index.html HTTP/1.1" 200 2326
EOF
```

第二步：基本字符串匹配（不加正则）

```
grep "192.168.1.10" ~/shell_practice/access.log # 查找包含该 IP 的行
```

✅ 输出该 IP 的两条访问记录

第三步：基础正则 — 匹配固定字符串

```
grep "HTTP/1.1" ~/shell_practice/access.log # 查找所有 HTTP 1.1 请求
grep "200" ~/shell_practice/access.log # 查找所有成功响应
```

✅ 能看到状态码 200（成功）和非 200 的区别

第四步：基础正则 — 字符类 []

```
grep "GET" ~/shell_practice/access.log # 查找所有 GET 请求
grep "[GET]" ~/shell_practice/access.log # 查找包含 G 或 E 或 T 的行（字符类是"或"的关系）
grep "20[01]" ~/shell_practice/access.log # 查找 200 或 201
```

✅ [GET] 和 GET 完全不同，注意区分

第五步：基础正则 — 锚点 ^ 和 \$

```
grep "^192" ~/shell_practice/access.log # 查找以 192 开头的行
grep "200$" ~/shell_practice/access.log # 查找以 200 结尾的行
grep "^10.0.0" ~/shell_practice/access.log # 查找以 10.0.0 开头的 IP 段
```

✅ ^ 锚定行首，\$ 锚定行尾，是正则最常用的定位符号

第六步：基础正则 — 元字符 . (任意单个字符)

```
grep "192.168.1.1." ~/shell_practice/access.log      # . 匹配任意单个字符
grep "10.0.0.8." ~/shell_practice/access.log        # 匹配 10.0.0.80 ~ 10.0.0.89
```

✅ `.` 是正则最常用的万能占位符

- 【收获】 `^` 行首锚定, `$` 行尾锚定, `[]` 字符类, `.` 任意单字符
- 【意义】 日志分析、配置查找时, 正则比普通 `grep` 精确得多

练习2：量词 —— *, +, ?

【生产环境使用场景】
从日志或配置文件中提取手机号、邮箱、IP 范围等, 需要匹配"出现多次"或"出现零次或一次"的情况。
量词说明:

量词	含义	匹配示例
*	前一个字符出现 0次或多次	10* → 1, 10, 100000
+	前一个字符出现 1次或多次	10+ → 10, 100000 (不含单独的 1)
?	前一个字符出现 0次或1次	10? → 1, 10
{n,m}	前一个字符出现 n 到 m 次	10{2,4} → 100, 1000, 10000

第一步：使用 + (1次或多次)

```
# 在基本正则中, grep 使用 -E 开启扩展正则
grep -E "10+" ~/shell_practice/access.log          # 匹配 10, 100, 1000... (10 出现1次或多次)
grep -E "[0-9]+" ~/shell_practice/access.log       # 匹配任意数字序列
```

✅ `-E` 表示使用扩展正则表达式 (ERE), `+` 才有特殊含义

第二步：使用 ? (0次或1次)

```
grep -E "10?0" ~/shell_practice/access.log        # 匹配 "100" 或 "10" 后跟 "0"
# 即: 匹配 "100" 或 "1000"... 中的 "100" 部分
```

第三步：使用 {n,m} (指定次数)

```
grep -E "[0-9]{3}" ~/shell_practice/access.log     # 匹配任意3位数字
grep -E "[0-9]{1,3}" ~/shell_practice/access.log   # 匹配1到3位数字
```

第四步：综合应用 —— 提取 IP 地址

```
# 匹配 IP 地址格式 (4段数字, 每段1-3位)
grep -E "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}" ~/shell_practice/access.log
```

第五步：综合应用 —— 提取 HTTP 状态码

```
grep -E "\"" [0-9]{3} " ~/shell_practice/access.log # 匹配 HTTP 响应码（引号空格+3 位数字+空格）
```

【收获】 **+** 最常用（1次或多次），**?** 次之（0或1次），***** 易误用（可匹配0次）

【意义】 提取手机号/邮箱/账号时，量词决定能否匹配成功

练习3：grep 常用选项与实战技巧

【生产环境使用场景】

在真实日志分析中，经常需要大小写不敏感查找、带行号显示、反向排除、多文件批量搜索。

准备：再创建一个配置文件用于练习

```
cat > ~/shell_practice/app.conf << 'EOF'
server_port=8080
server_name=localhost
max_connections=100
timeout=300
LOG_LEVEL=INFO
log_file=/var/log/app.log
EOF
```

选项一：-i 忽略大小写

```
grep -i "log" ~/shell_practice/app.conf # 匹配 log, LOG, Log, LoG...
```

选项二：-n 显示行号

```
grep -n "server" ~/shell_practice/app.conf # 显示匹配行号，方便定位
```

选项三：-v 反向排除

```
grep -v "^#" ~/shell_practice/app.conf # 排除注释行（以 # 开头的行）
grep -v "^$" ~/shell_practice/app.conf # 排除空行
grep -v "^#" ~/shell_practice/app.conf | grep -v "^$" # 组合：去掉注释和空行
```

选项四：-c 统计匹配行数

```
grep -c "200" ~/shell_practice/access.log # 统计状态码为 200 的请求数量
grep -c "HTTP" ~/shell_practice/access.log # 统计总请求数
```

选项五：-o 只输出匹配部分

```
grep -oE "[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}"
~/shell_practice/access.log
# 只输出 IP 地址本身，不输出整行
```

选项六：-r 递归搜索目录下所有文件

```
grep -r "ERROR" ~/shell_practice/
```

```
# 在整个练习目录中搜索 ERROR
```

【收获】 `-i` 忽略大小写，`-v` 反向排除，`-n` 显示行号，`-c` 统计行数，`-o` 只输出匹配部分

【意义】 组合使用这些选项，是日志分析的标准操作

练习4：sed —— 替换文本

【生产环境使用场景】

批量修改配置文件中的参数（如换端口号、改日志路径）、替换日志中的敏感信息、删除特定行——sed 是流编辑器，能在不打开文件的情况下完成修改，是自动化脚本的核心工具。

基本语法：

```
sed [选项] 's/要查找的内容/替换后的内容/[标志]' 文件名
```

`s` 表示 substitute（替换），是最常用的 sed 命令

第一步：基本替换

```
sed 's/8080/9090/' ~/shell_practice/app.conf # 把第一行的 8080 替换为 9090  
（默认只换每行第一个）
```

✓ 注意：默认只替换**每行第一个**匹配项，原文件不变（需要加 `-i` 才真正写入）

第二步：全局替换（最常用）

```
sed 's/8080/9090/g' ~/shell_practice/app.conf # g 表示 global，每行所有匹配项  
都替换
```

✓ 工作中 `s/xxx/yyy/g` 是 sed 最常用的形式，**务必记住加 g**

第三步：-i 真正写入文件

```
# 备份后再修改（推荐）  
sed -i.bak 's/8080/9090/g' ~/shell_practice/app.conf # 修改并备份原文件为 .bak  
cat ~/shell_practice/app.conf # 查看修改后的文件
```

✓ `-i` 直接修改文件内容，使用前建议先备份

第四步：使用正则匹配

```
# 把所有数字端口号替换为 8000  
sed -E 's/[0-9]+/8000/g' ~/shell_practice/app.conf # -E 开启扩展正则，+ 才生效
```

第五步：删除特定行

```
sed '/^#/d' ~/shell_practice/app.conf # 删除所有注释行（以 # 开头）  
sed '/^$/d' ~/shell_practice/app.conf # 删除所有空行  
sed '/^#/d; /^$/d' ~/shell_practice/app.conf # 删除注释行和空行
```

第六步：在多个文件中批量操作

```
sed -i.bak 's/INFO/WARN/g' ~/shell_practice/*.conf # 批量替换所有 .conf 文件
```

【收获】 `sed -i.bak 's/原内容/新内容/g' file` — 这是 sed 实战最标准写法

【意义】 批量替换配置参数、清理日志文件、自动化部署脚本中都会用到 sed

练习5：awk —— 提取字段

【生产环境使用场景】

日志文件通常以固定分隔符（如空格、逗号）组织字段，awk 能按列提取数据，比 cut 更强大。常用于统计访问量、计算总和、格式化输出。

基本语法：

```
awk '{ print $1, $2 }' 文件名
```

`$0` = 整行, `$1` = 第1列, `$2` = 第2列, `$NF` = 最后一列 (NF = Number of Fields)

第一步：按分隔符提取列

```
# /etc/passwd 以 : 分隔, 提取第1列 (用户名) 和第5列 (用户信息)
awk -F: '{ print $1, $5 }' /etc/passwd | head -5
```

✓ `-F:` 指定冒号为分隔符, 默认以空格/tab 分隔

第二步：分析 nginx 访问日志 (提取 IP 和状态码)

```
awk '{ print $1, $9 }' ~/shell_practice/access.log
```

✓ `$1` = IP 地址, `$9` = HTTP 状态码 (200/401/403)

第三步：统计每个 IP 的访问次数

```
awk '{ print $1 }' ~/shell_practice/access.log | sort | uniq -c
```

✓ `sort | uniq -c` 统计每个 IP 出现次数

第四步：统计每种 HTTP 状态码的出现次数

```
awk '{ print $9 }' ~/shell_practice/access.log | sort | uniq -c
```

✓ 能看到 200/401/403 各出现几次

第五步：计算总和

```
# 计算 access.log 总行数
awk 'END { print NR }' ~/shell_practice/access.log
```

✓ `NR` = Number of Records (总行数), `END` 是处理完所有行后执行

第六步：格式化输出

```
# 格式化输出 IP 和访问次数，标题友好
awk '{ count[$1]++ } END { for (ip in count) print ip, count[ip] }'
~/shell_practice/access.log
```

✅ 用关联数组统计，END 块输出最终结果

【收获】 `awk -F: '{ print $1, $NF }'` 按列提取，`$NF` 永远是最后一列

【意义】 日志统计、CSV 数据处理、格式化报表是运维和数据分析的日常

练习6：Shell 三目运算符

【生产环境使用场景】

当 if 判断只有简单两种情况（真/假），且每个分支只有一条命令时，三目运算符比 if 更简洁。常用于变量赋值默认值、根据条件给变量赋不同值等场景。

基本语法：

```
[ 条件 ] && 真时执行的命令 || 假时执行的命令
```

等价于：

```
if [ 条件 ]; then 真时执行的命令; else 假时执行的命令; fi
```

第一步：最简形式——设置默认值

```
# 如果变量为空，则设置为默认值
NAME=""
[ -z "$NAME" ] && NAME="Guest" || NAME="$NAME"
echo $NAME                                     # 输出: Guest
```

✅ `-z` 判断字符串为空时为真，`&&` 执行左边，`||` 执行右边

三目运算符的标准形式（推荐）：

```
# 如果 NAME 为空，则设为 Guest
NAME=""
NAME=[ -z "$NAME" ] && echo "Guest" || echo "$NAME"
# 或者直接赋值：
NAME="" && true || echo "Guest"
```

更实用的写法：

```
# 用 ${变量:-默认值} 更简洁（推荐）
NAME=""
echo "${NAME:-Guest}"                                     # 输出: Guest
echo "${NAME:-default}"                                   # 原来 NAME 不变，只是 echo 时用默认
值
```

第二步：根据条件赋值

```
# 判断文件是否存在，存在则显示 "存在"，否则显示 "不存在"
FILE="/etc/passwd"
[ -f "$FILE" ] && echo "存在" || echo "不存在"
```

✅ `-f "$FILE"` 判断文件是否存在

第三步：替代简单的 if 判断

```
# if 判断（较啰嗦）：
if [ $AGE -ge 18 ]; then
    echo "成年"
else
    echo "未成年"
fi

# 三目运算符（简洁）：
AGE=20
[ $AGE -ge 18 ] && echo "成年" || echo "未成年"
```

✅ 只适合“真/假各一条命令”的简单场景，复杂场景仍用 if

第四步：整数大小比较

```
A=10
B=20
[ $A -lt $B ] && echo "$A < $B 成立" || echo "不成立"
```

✅ `-lt` = less than, `-gt` = greater than

第五步：判断目录是否存在，不存在则创建

```
DIR="/tmp/testdir"
[ -d "$DIR" ] && echo "目录已存在" || mkdir -p "$DIR"
```

✅ `-d` 判断目录是否存在

【收获】 `[ 条件 ] && 命令1 || 命令2` — 适合简单的“真/假各一条命令”的判断

【意义】 变量默认值设置、简单条件输出、脚本中的快速分支选择，比 if 更简洁

第6章 核心命令表

命令/语法	作用	关键选项
<code>grep "正则" 文件</code>	按正则匹配行	<code>-i</code> 忽略大小写 <code>-v</code> 反向 <code>-n</code> 行号 <code>-c</code> 计数 <code>-o</code> 只输出匹配 <code>-E</code> 扩展正则
<code>grep -E "扩展正则" 文件</code>	扩展正则匹配	<code>-E</code> 开启 ERE, <code>+ ? {n,m}</code> 才生效
<code>sed 's/原/新/g' 文件</code>	替换文本	<code>-i.bak</code> 写入并备份, <code>-E</code> 扩展正则
<code>sed '/关键字/d' 文件</code>	删除匹配行	<code>^#</code> 注释行, <code>^\$</code> 空行

命令/语法	作用	关键选项
<code>awk '{print \$1,\$NF}'</code> 文件	按列提取	<code>-F</code> : 指定分隔符, <code>\$NF</code> 最后一列
<code>awk 'END{print NR}'</code> 文件	统计/汇总	<code>NR</code> 总行数, <code>\$0</code> 整行
<code>\${var:-默认值}</code>	变量为空时 默认值	最简洁的默认值写法
<code>[条件] && cmd1 \ \ cmd2</code>	三目运算符	适合简单的真/假单命令分支

第6章 常见问题

- **Q: grep 正则没生效** → 确认是否加了 `-E` (扩展正则), 否则 `+ ? { }` 会被当作普通字符
- **Q: sed 替换没反应** → 检查是否漏了 `g` (全局标志), 没加 `g` 只替换每行第一个
- **Q: sed -i 改错了** → 加上 `.bak` 后缀自动备份, 或先用 `sed` 不加 `-i` 预览结果
- **Q: awk 提取的列不对** → 确认分隔符是否正确, 日志中多个空格会导致列错位, 可用 `-F'[ ]+'` 合并连续空格
- **Q: 三目运算符结果不对** → 如果 `&&` 部分命令执行失败 (返回非0), `\|\|` 也会被执行; 复杂判断仍用 `if`
- **Q: 正则中 . 匹配不到** → `.` 默认匹配任意单个字符, 不需要转义; 但在 `grep -E` 中 `.` 同样匹配任意字符, `\.` 才表示字面点号